



初试Java模块化开发

北京理工大学计算机学院
金旭亮

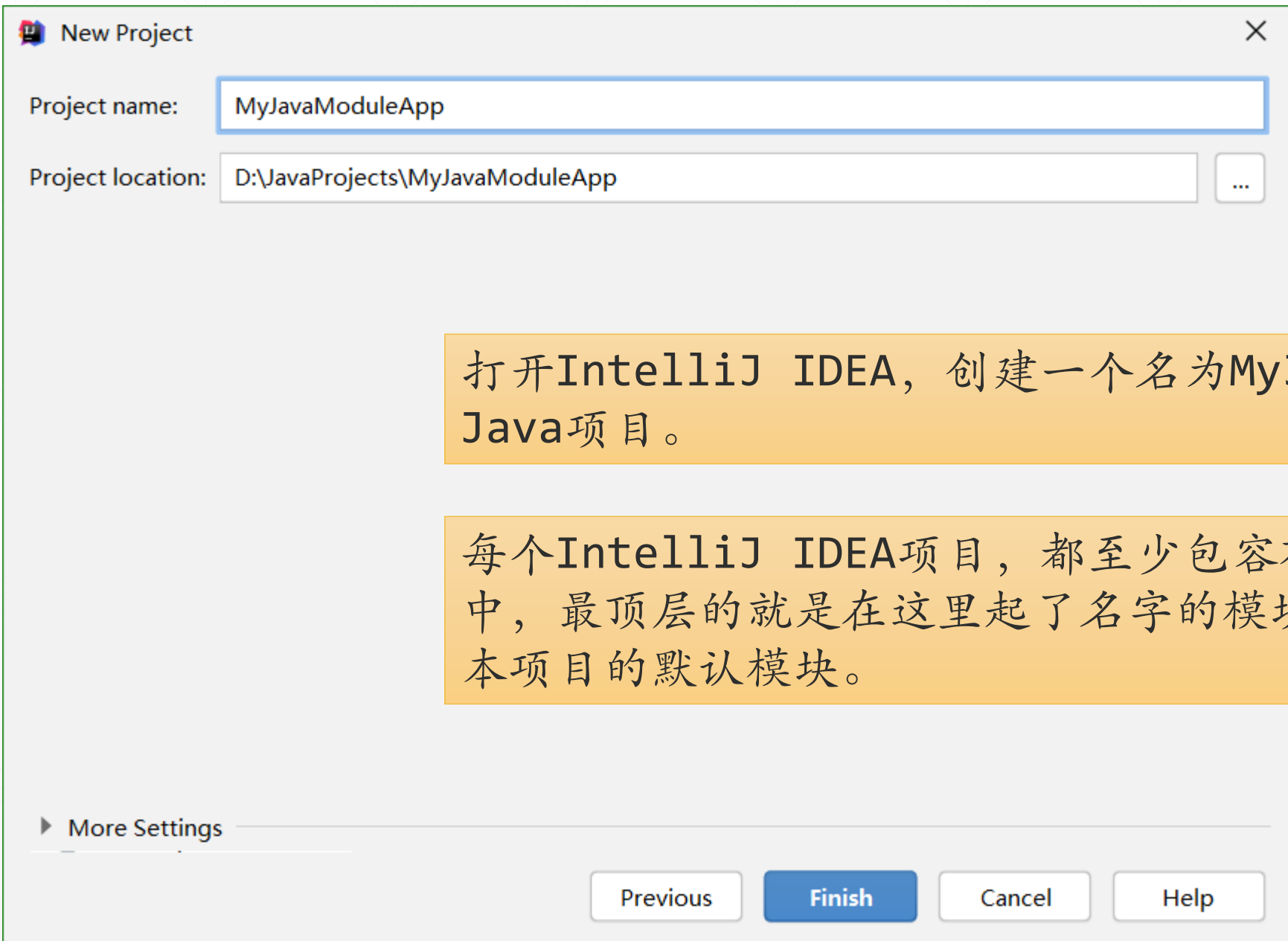
概述



前面我们已经介绍了Java模块化的基础知识，在本讲中，我们将使用IntelliJ IDEA创建一个简单的模块化Java应用，通过这个例子，和前面课程所介绍的内容相对照，同学们就能对Java模块有深入的认识。

这里要特别注意一下，IntelliJ IDEA项目基本的构建单元也叫“模块（Module）”，这里的“模块”，与JPMS中的“模块”不是一回事。IntelliJ中的模块是“构建单元”，它可以用于组织多种类型的开发资源。在实际开发中，可以使用IntelliJ模块来创建和生成JPMS模块。

本讲示例：MyJavaModuleApp

The image shows the 'New Project' dialog box in IntelliJ IDEA. The dialog has a title bar with the IntelliJ logo and the text 'New Project'. It contains two input fields: 'Project name:' with the value 'MyJavaModuleApp' and 'Project location:' with the value 'D:\JavaProjects\MyJavaModuleApp'. There is a 'More Settings' link at the bottom left and four buttons at the bottom right: 'Previous', 'Finish', 'Cancel', and 'Help'.

New Project

Project name: MyJavaModuleApp

Project location: D:\JavaProjects\MyJavaModuleApp

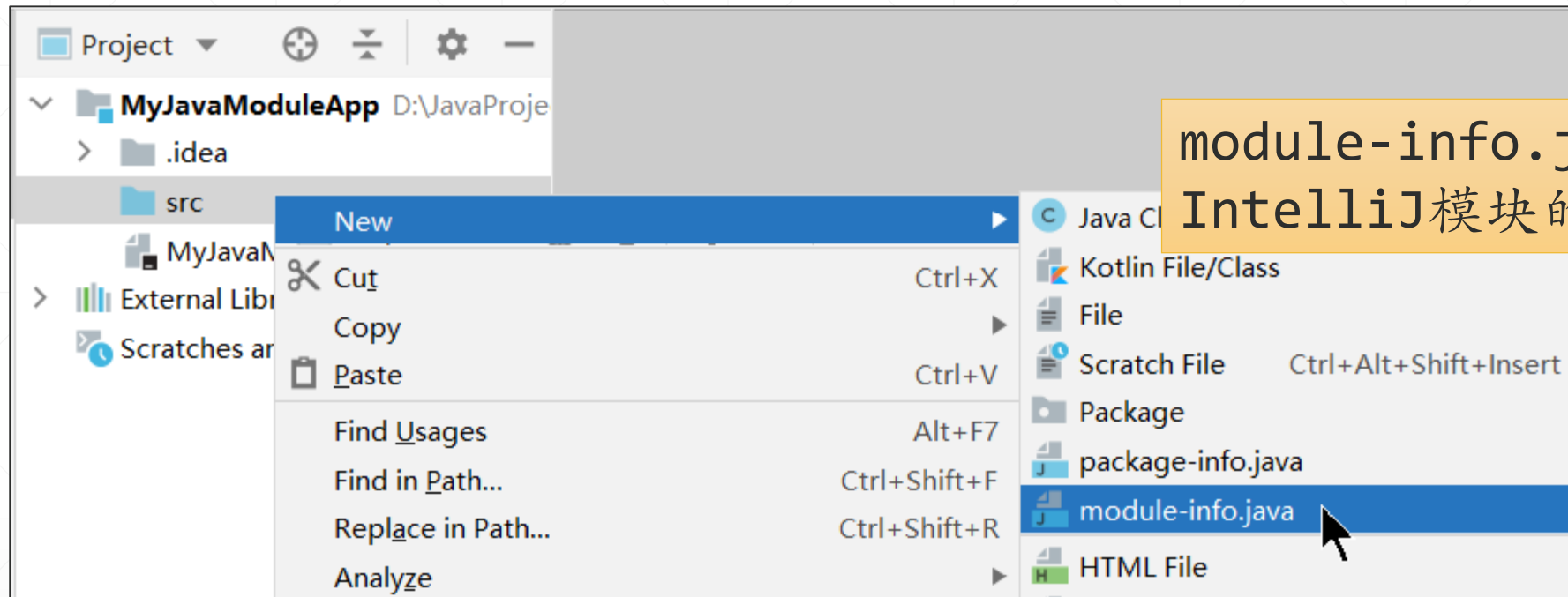
More Settings

Previous Finish Cancel Help

打开IntelliJ IDEA，创建一个名为MyJavaModuleApp的Java项目。

每个IntelliJ IDEA项目，都至少包容有一个模块，在本例中，最顶层的就是在这里起了名字的模块，可以将它看成是本项目的默认模块。

向示例项目中添加“模块描述符”文件

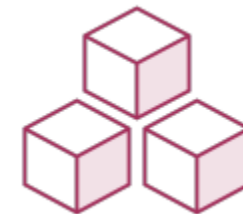


module-info.java文件应该位于IntelliJ模块的src文件夹下。

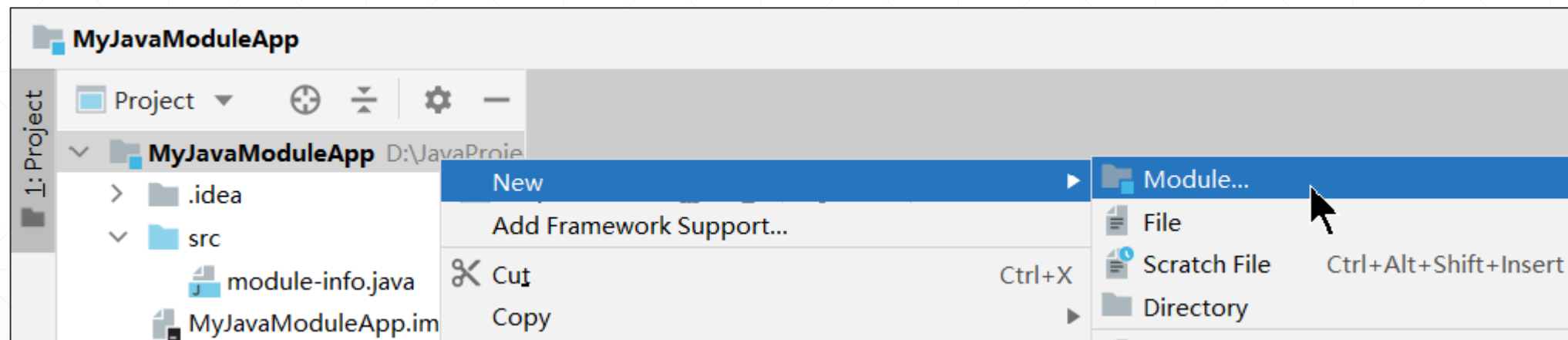


module之后，跟着的就是模块名，由于所有模块都位于一个全局命名空间中，因此，模块名称必须是唯一的，并且应该与模块文件夹的名字相同。

创建多模块的IntelliJ项目

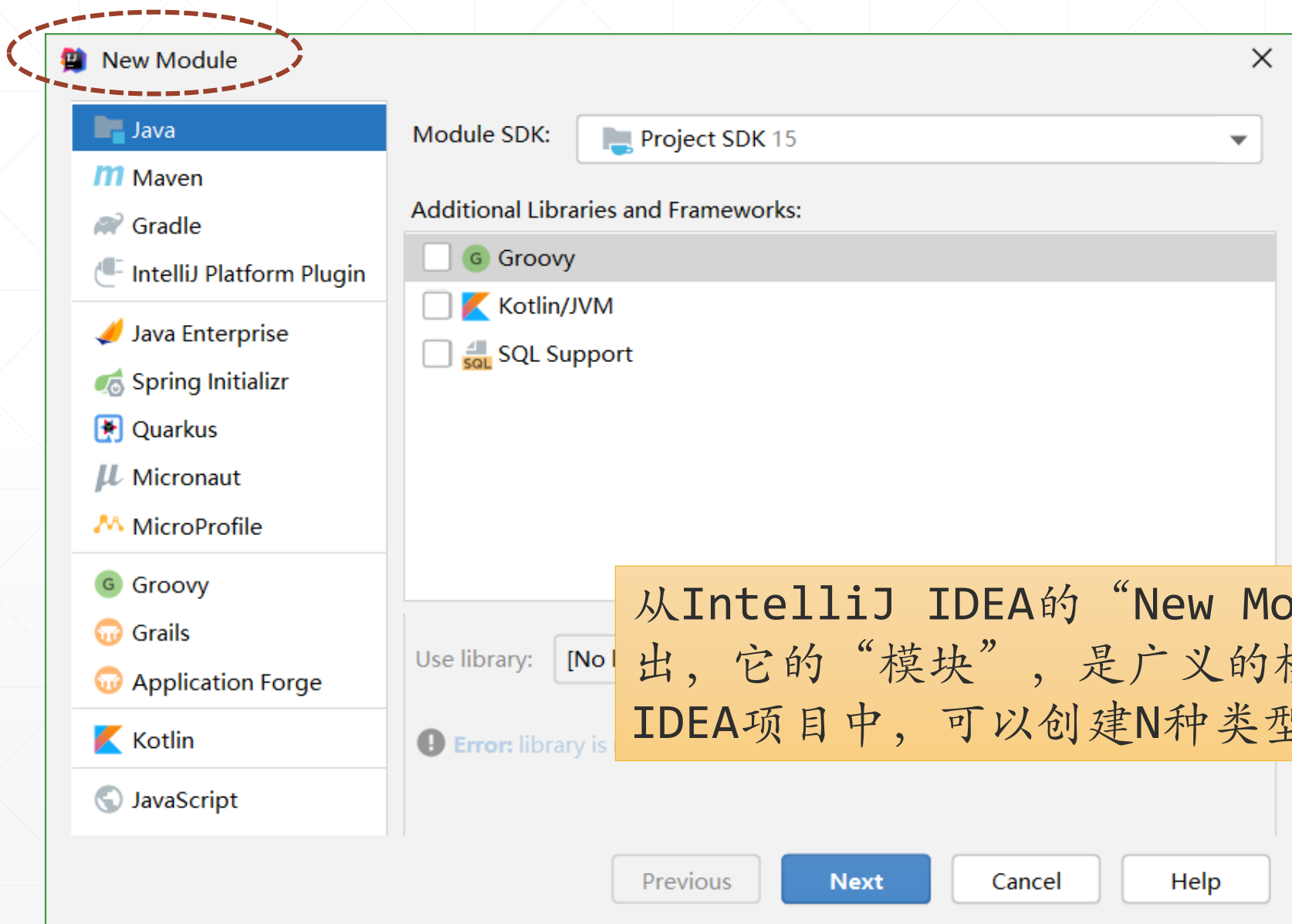


向IntelliJ项目添加新模块：

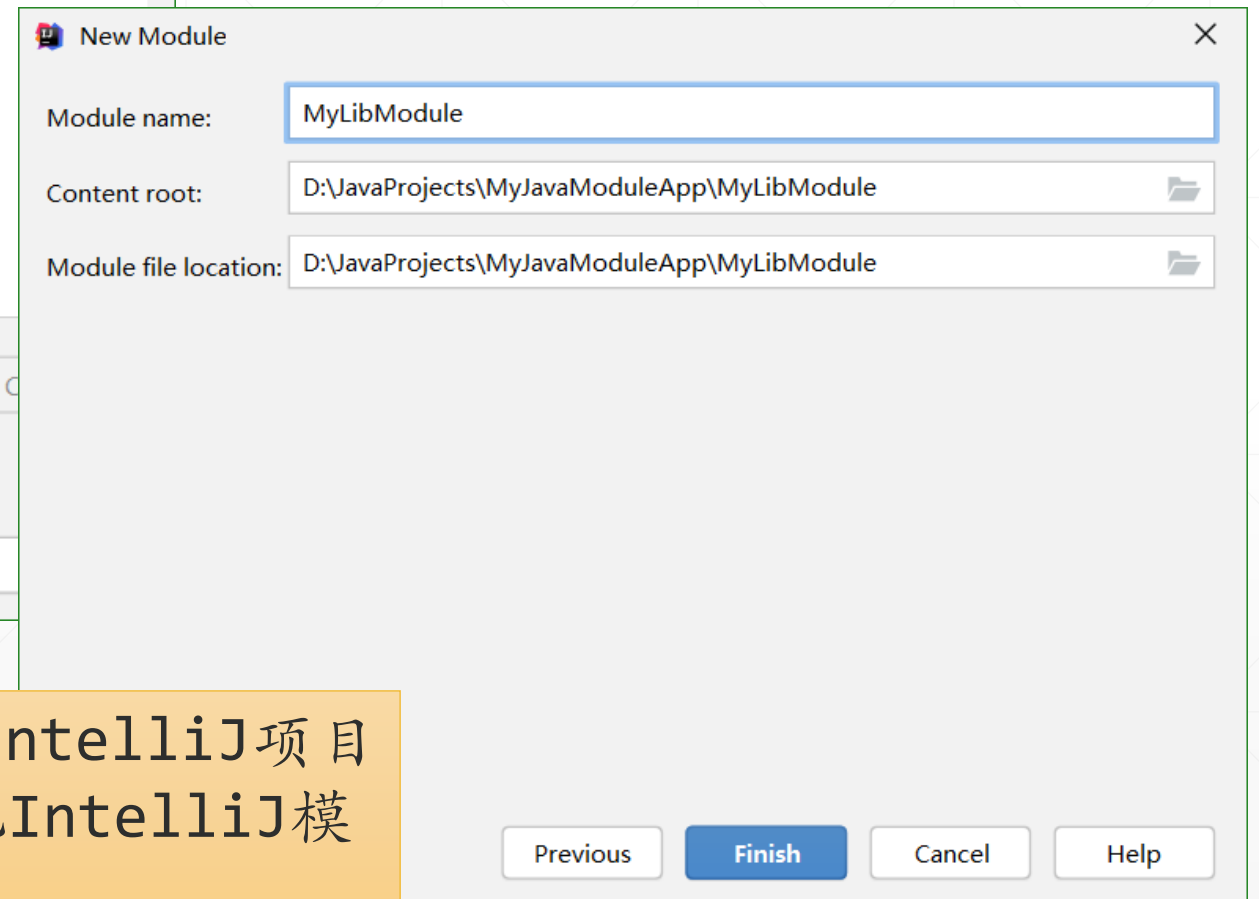
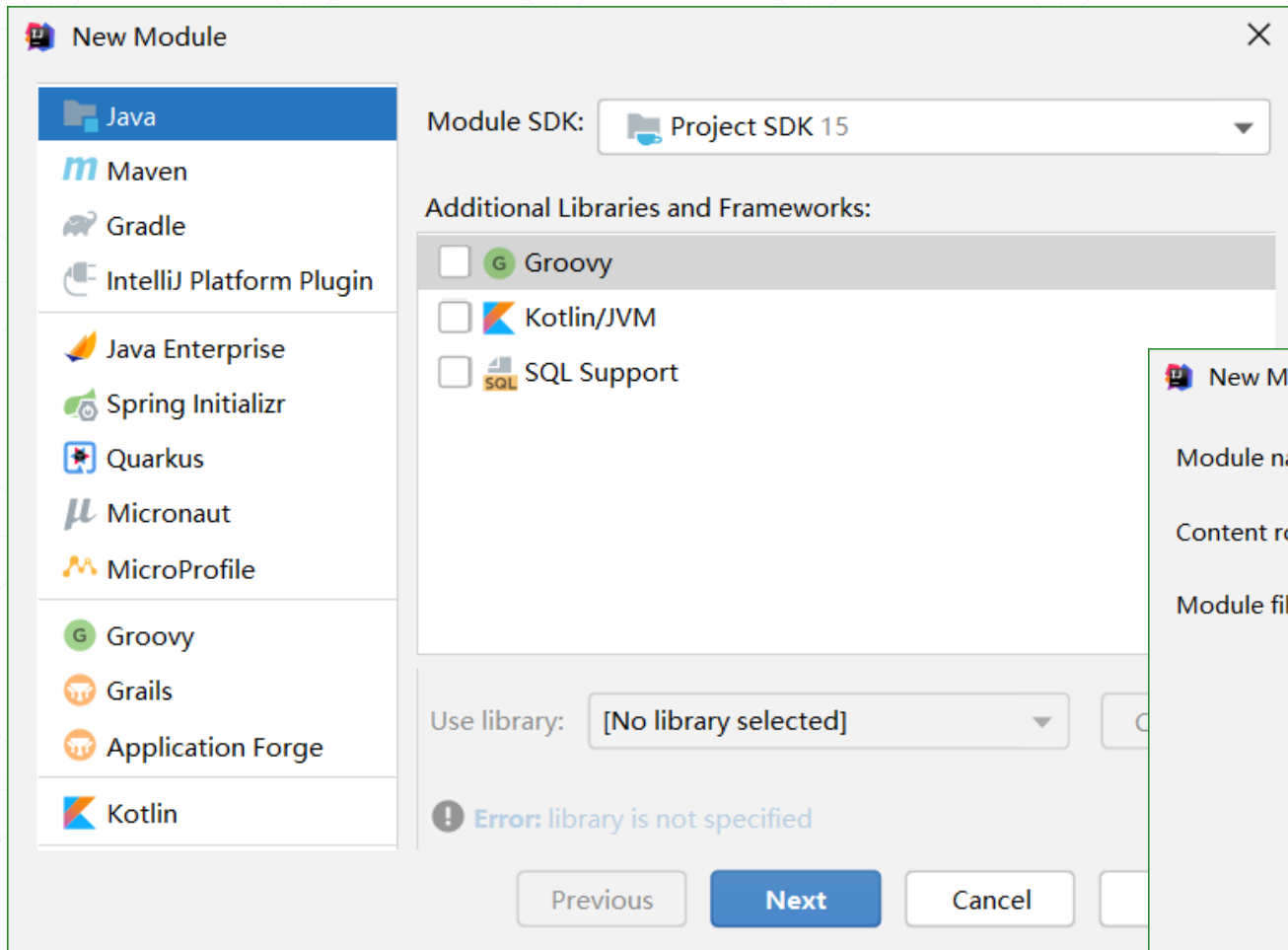


- 一个IntelliJ项目可以包容多个模块，这在实际开发中非常常见。
- 通常会将代码等相关资源按照“职责”进行分类和汇总，把密切相关的东西都放在一起，保存在某个模块中，然后，再建立起各个模块之间的依赖关系。
- 达到一定规模的Java项目，不应该将所有的东西全部放在一个模块中。

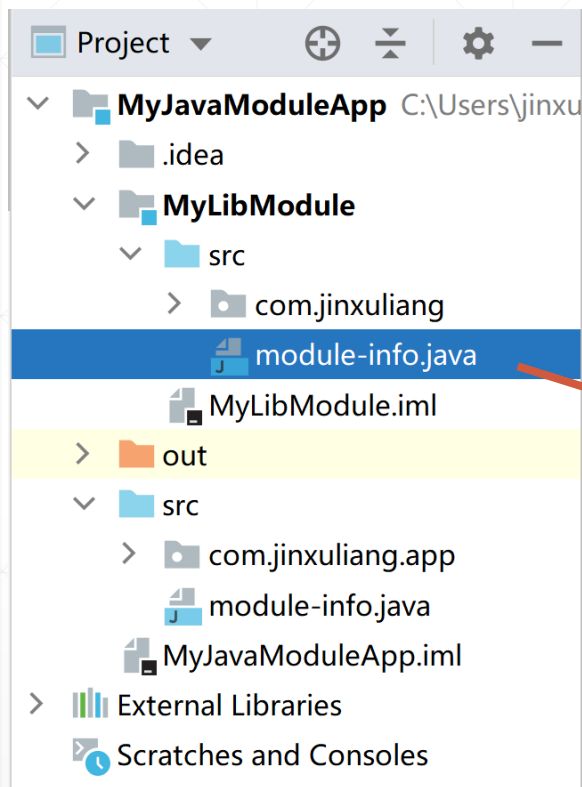
IntelliJ项目添加“新模块”对话框



从IntelliJ IDEA的“New Module”对话框可以看出，它的“模块”，是广义的模块，在一个IntelliJ IDEA项目中，可以创建N种类型的“模块”。



创建IntelliJ模块的过程，与创建一个IntelliJ项目的过程几乎一样，这也说明了，你可以把IntelliJ模块，看成是一个IntelliJ的“子项目”。



模块创建好之后，采用与前面一样的方法，为新模块添加module-info.java文件

```
module-info.java x
1 //模块中可以放置的内容:
2 // (1) 声明本模块依赖的其他模块 (如果有的话)
3 // (2) 声明本模块导出的包
4 module MyLibModule {
5 }
```



每个模块，都要有自己的module-info.java。



模块名称必须是唯一的。

模块声明与模块描述符



module-info.java称为“**模块声明 (Module Declaration) 文件**”，它编译之后，称为“**模块描述符 (Module Descriptor)**”。



模块声明文件必须存在于模块目录的根目录中。它与其他源文件一起编译成一个名为module-info.class的二进制类文件。



如果使用javac一次性编译多模块的Java模块化项目中，模块名称必须与包含模块描述符的目录名称相匹配。否则，编译器将拒绝编译并报告匹配错误。

模块化的JAR包

module-info.java

模块声明文件



module-info.class

模块描述符
(module descriptor)



包容有模块描述符的JAR包，称为**模块化JAR包**。



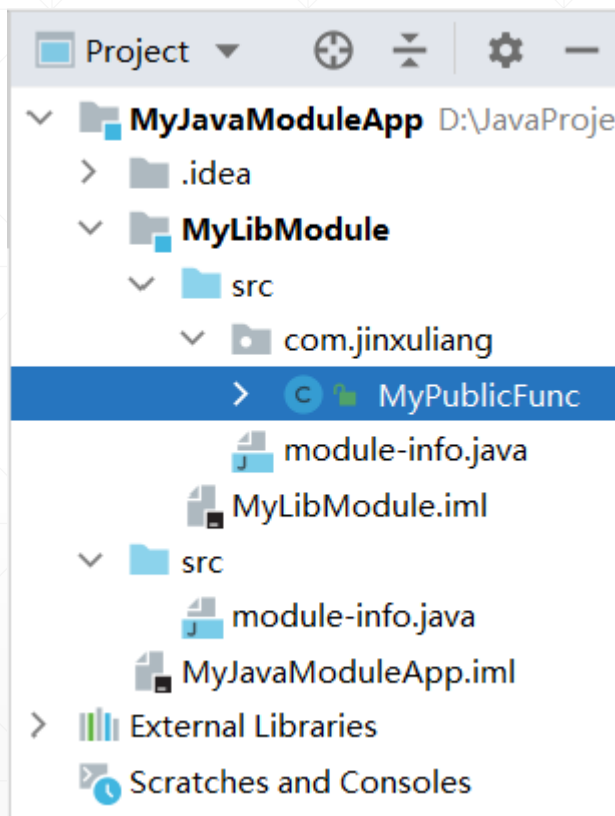
模块描述符必须放在模块化JAR包的根目录下，
以方便JPMS识别。



模块化JAR可以当成普通的JAR来使用，你只需要
把它放到类路径中去就行了。

给MyLibModule模块添加可复用的代码

向MyLibModule模块中添加一个包，再添加一个类，然后里面写了一个getInfo()方法，返回一个“提示信息”。



```
MyPublicFunc.java x
1 package com.jinxuliang;
2
3 public class MyPublicFunc {
4     @ public static String getInfo(){
5         return "MyLibModule模块中的MyPublicFunc.getInfo()方法被调用";
6     }
7 }
```

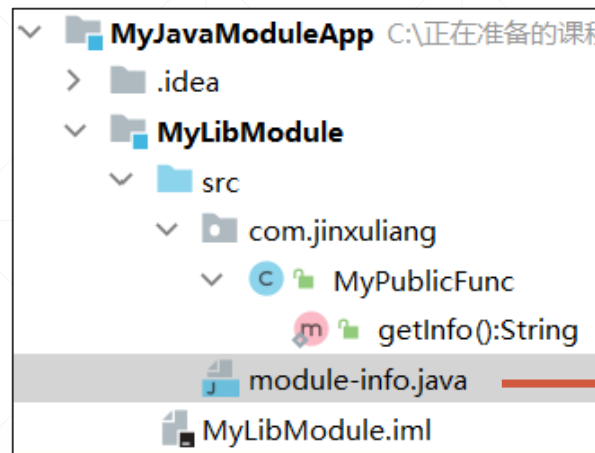
一会儿将在另一个模块中调用这个方法



模块化的应用中，所有的类，都应该归属于一个有名字的包。

修改MyLibModule模块的module-info.java

在module-info.java文件中，使用exports导出“com.jinxuliang”包。



```
1 // 模块中可以放置的内容:
2 // (1) 声明本模块依赖的其他模块 (如果有的话)
3 // (2) 声明本模块导出的包
4 module MyLibModule {
5     exports com.jinxuliang;
6 }
```

MyLibModule模块导出了一个包，此包中所有的公有类（本例为MyPublicFunc）现在就可以被其他模块所使用。简单地说，只要在module-info.java文件中导出一个包之后，包中所有声明为public的类，就可以被外部所访问了。



在模块化的Java应用中，如果没有在module-info.java文件中导出包，那么，包中的类即使被声明为public，外界也无法使用它。

总结一下模块描述符的职责



所谓“模块描述符”，其实就是放在`module-info.java`文件中的内容编译之后的结果（是一个`.class`文件），它描述了模块相关的重要信息：



模块的名字



本模块依赖于其他哪些模块的哪些包



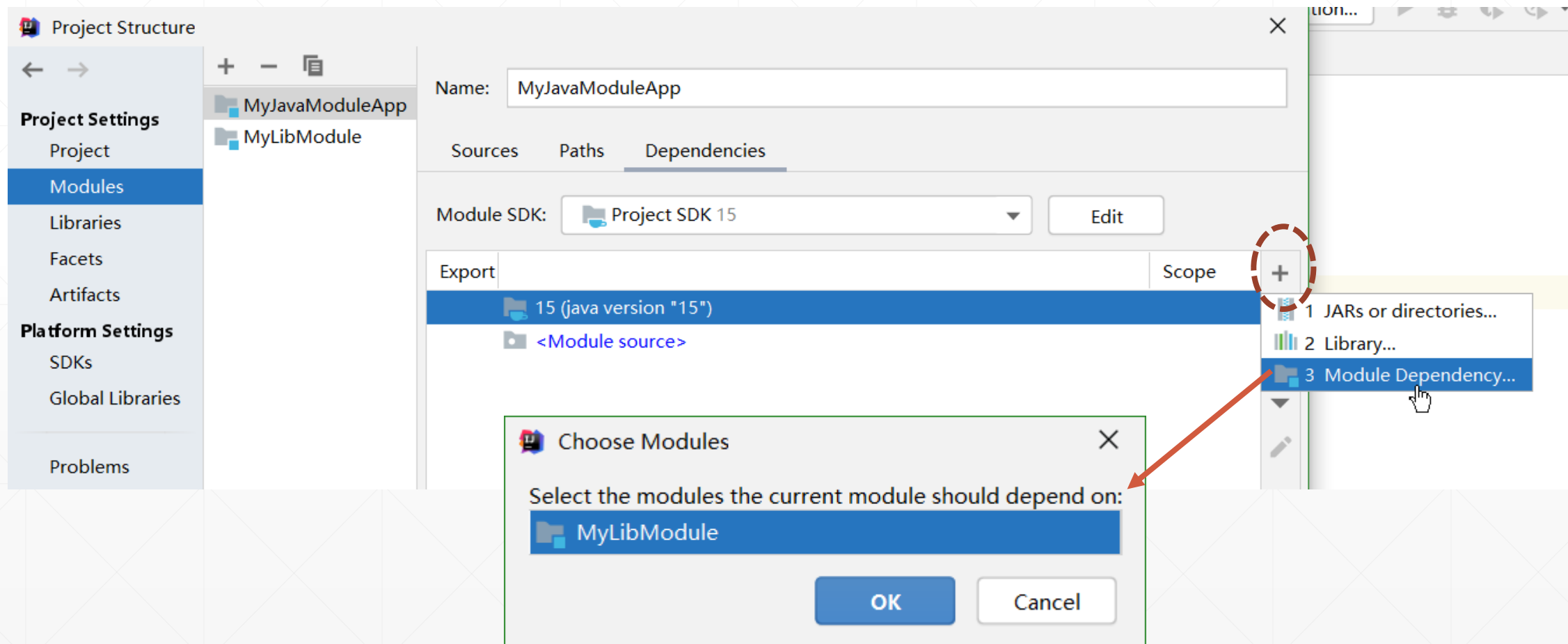
本模块导出了哪些包和服务

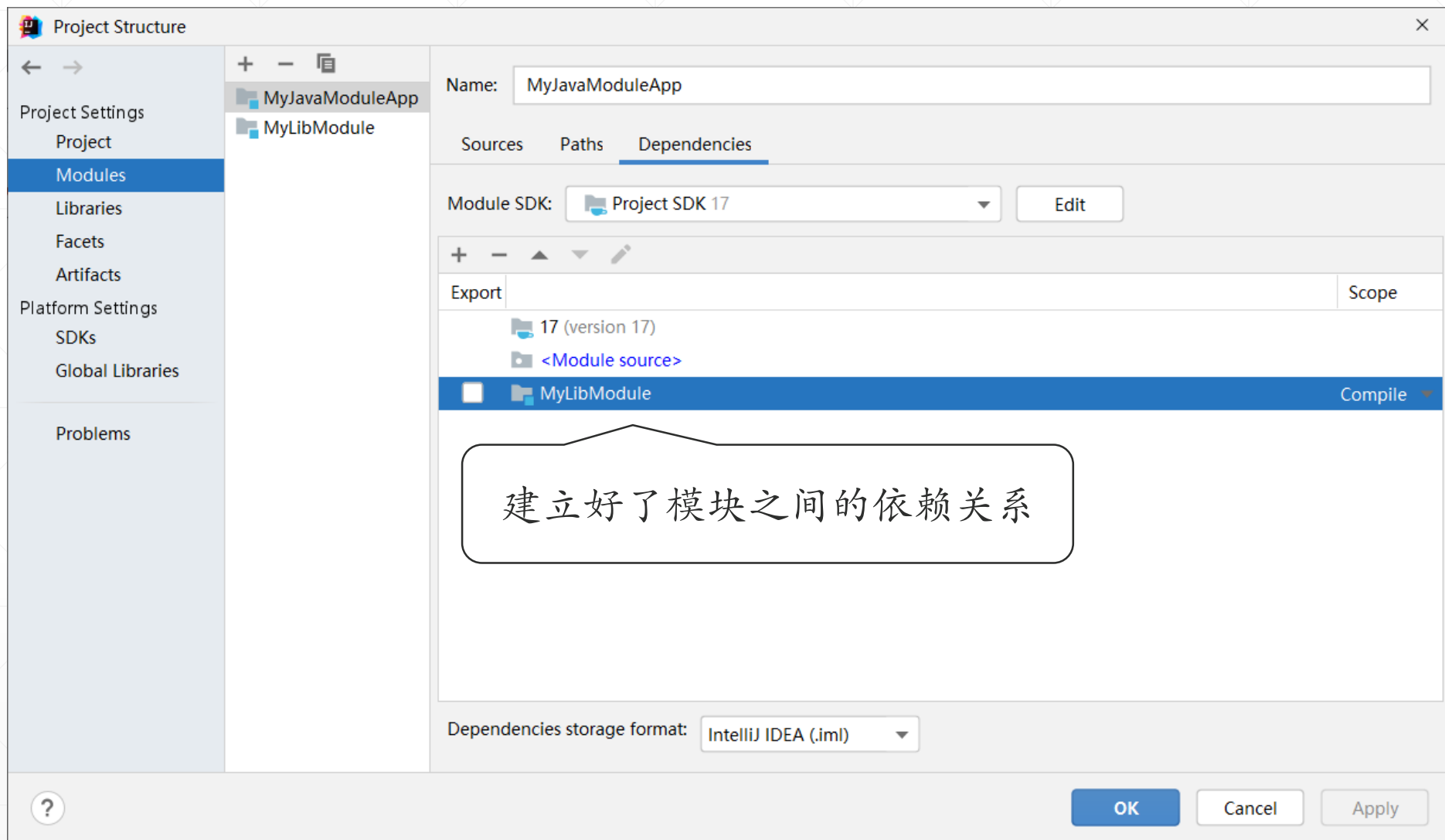


本模块是否允许外界使用反射，读取特定包中的类型。

建立模块间的依赖关系

File-->Project Structure





这里所建立的依赖关系，其实是编译时的依赖关系，仅在编译时起作用。

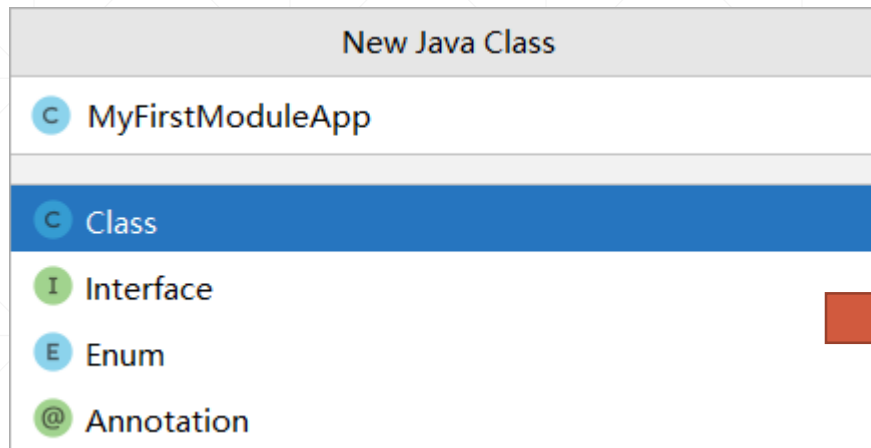
修改MyJavaModuleApp模块的module-info.java



```
1 //模块的名字, 其实就是IntelliJ IDEA的Project名
2 //因为IntelliJ中的项目, 至少都有一个模块 (称为默认模块)
3 module MyJavaModuleApp {
4     //声明本模块需要调用MyLibModule模块中的代码
5     requires MyLibModule;
6 }
```

- 使用requires语句导入本模块所需要调用的代码所在的模块。
- 如果本模块中也有代码需要供给外界使用, 则可以用exports语句导出。

编写测试代码测试模块的调用



```
MyFirstModuleApp.java x
1  import com.jinxuliang.MyPublicFunc;
2
3  public class MyFirstModuleApp {
4      public static void main(String[] args) {
5          //调用另一模块中的方法
6          System.out.println(MyPublicFunc.getInfo());
7      }
8  }
```

由于本模块使用requires关键字导入了MyLibModule模块，所以可以使用它所导出的MyPublicFunc类。



运行

Run: MyFirstModuleApp x

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:
MyLibModule模块中的MyPublicFunc.getInfo()方法被调用

Process finished with exit code 0
```

“模块路径”的概念



当编译和运行模块时，Java编译器和运行时使用模块描述符来解析正确的模块。模块是从**模块路径 (module path)** 中解析出来的，而不是**类路径 (class path)**。类路径用于构建编译项目所需要用到的类型清单，而模块路径则可以看出是一个项目所引用的模块的清单。



java命令行有一个module-path参数，用于指定模块路径。



当使用java命令运行一个模块时，它到模块路径中去查找所有模块，装载其模块描述符，通过读取模块描述符中给定的exports和requires等信息，就能构建出程序中用到的所有模块的依赖图。

给IntelliJ模块添加“模块依赖”时，其实是为了指定编译信息。真正的Java模块依赖信息，是放在module-info.java中的。

使用命令行运行模块化后的Java程序

- ✓ 如果初始模块中定义了主类：

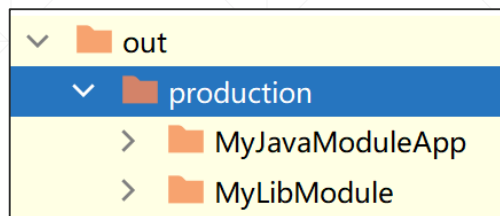
```
java  
--module-path 存放模块的文件夹列表  
--module 初始模块
```

使用—module指定的模块是初始模块，它是模块解析的起点。

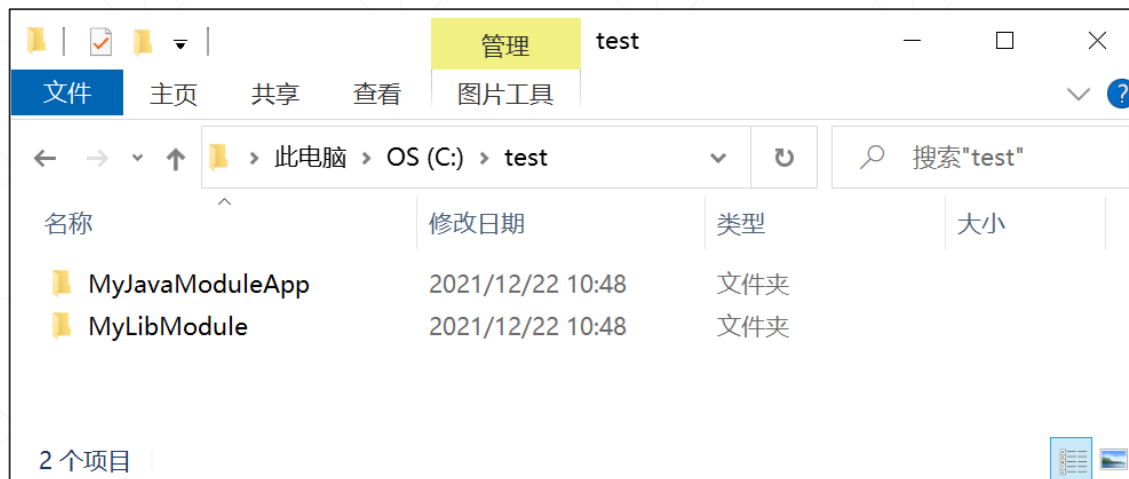
- ✓ 如果初始模块中没有定义主类，则需要运行时动态指定主类：

```
java  
--module-path 存放模块的文件夹列表  
--module 初始模块/包容包名的主类名
```

使用命令行运行模块实战演练



复制到



在命令行中运行:

```
Windows PowerShell
PS C:\test> java --module-path ".\MyJavaModuleApp;.\MyLibModule" -m MyJavaModuleApp/com.jinxuliang.app.MyFirstModuleApp
MyLibModule模块中的MyPublicFunc.getInfo()方法被调用
PS C:\test>
```



-m是“--module”的简写。

小结：使用命令行运行模块化Java应用的关键参数



`--module-path`，简写为 `-p`：代表依赖的模块保存的位置



`--module`，简写为 `-m`：表示要启动的初始模块，如果此模块中没有定义程序入口点，则可以采用“模块名/主类”的方式指定程序入口点。

```
java
  --module-path ".\mods;.\libs"
  --module mymainmodule/com.example.MainClass
```

小结：Java模块编写的“基本准则”



在module-info.java中：

导出 (exports) 一个包 (package)

需要 (requires) 一个模块 (module)

- 被导出包中的所有公有类，可以被其他模块所使用。
- 不在导出包中的类，即使是公有 (public) 的，其他模块也无法使用它。

区分两个“性”

- 可读性 (readability)

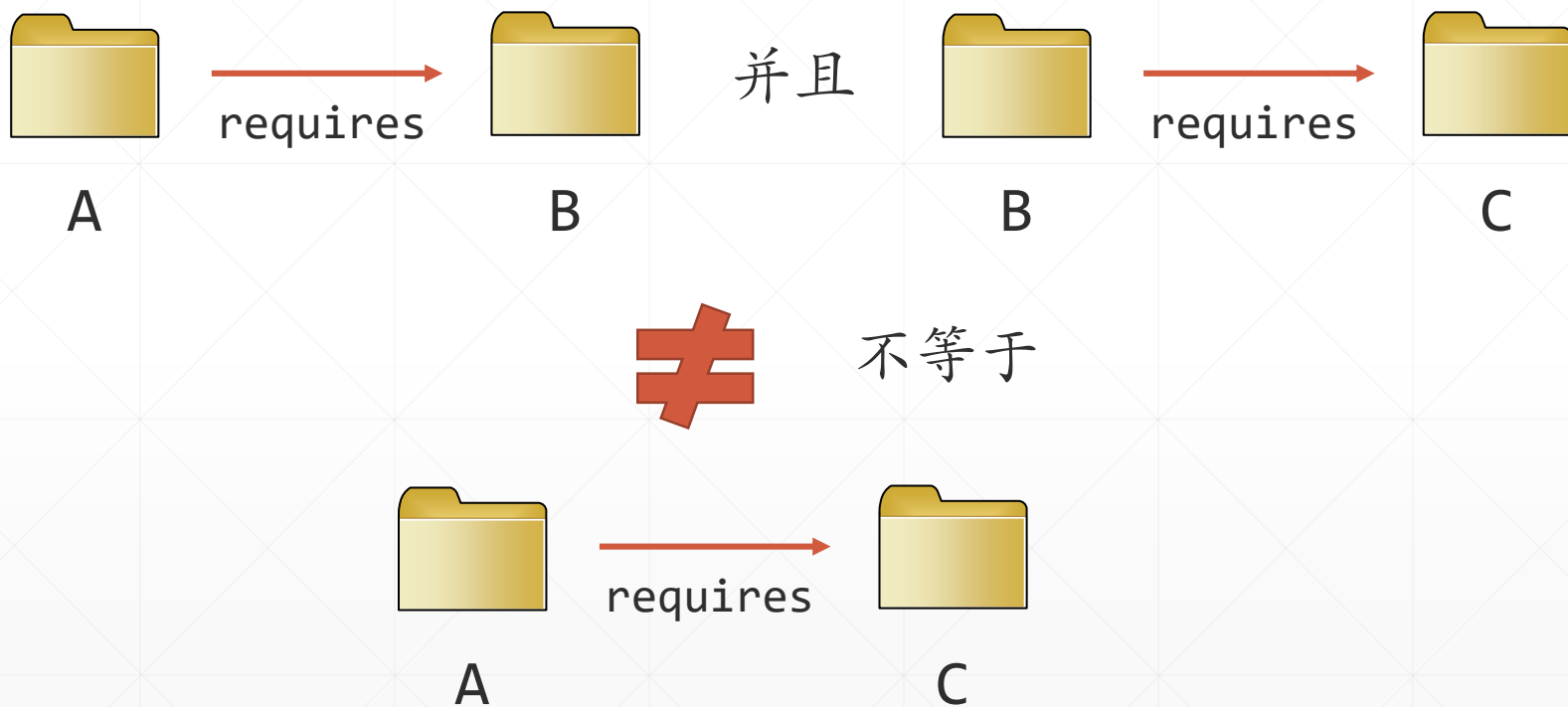
如果模块A `requires` 模块B，我们就说：“模块B相对于模块A来说，是可读的。读取其他模块意味着可以访问其导出包中的类型。

- 可访问性

访问修饰符	类	包	子类	无限制
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	
<code>- (default)</code>	✓	✓		
<code>private</code>	✓			

同时满足这两个条件，特定模块中的代码，才可以访问另一模块中的类型。

默认情况下，模块可读性必须显式声明，不能传递



区分编译时和运行时



编译时和运行时

`exports <--> requires`

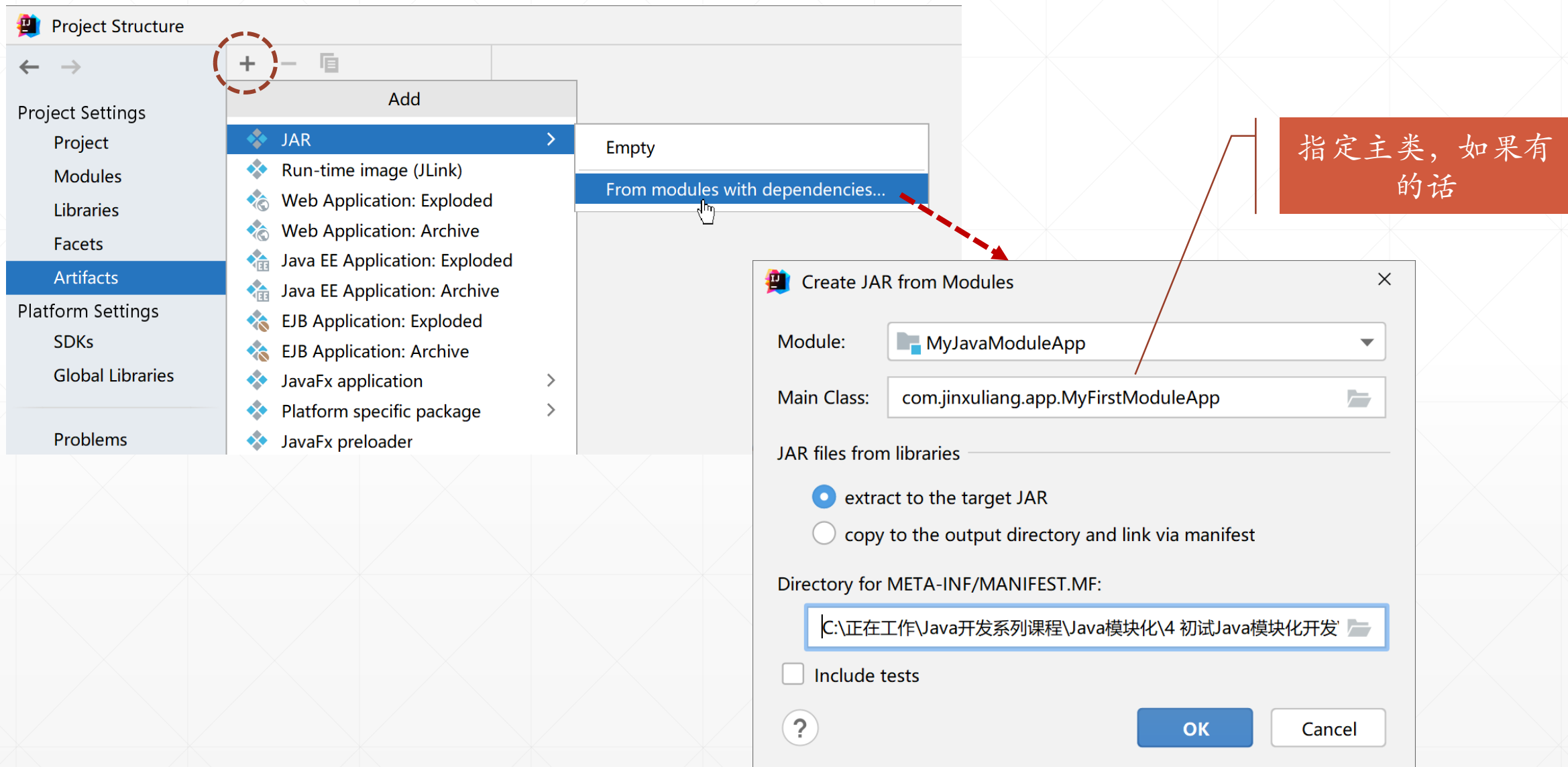
运行时

`uses <--> provides`

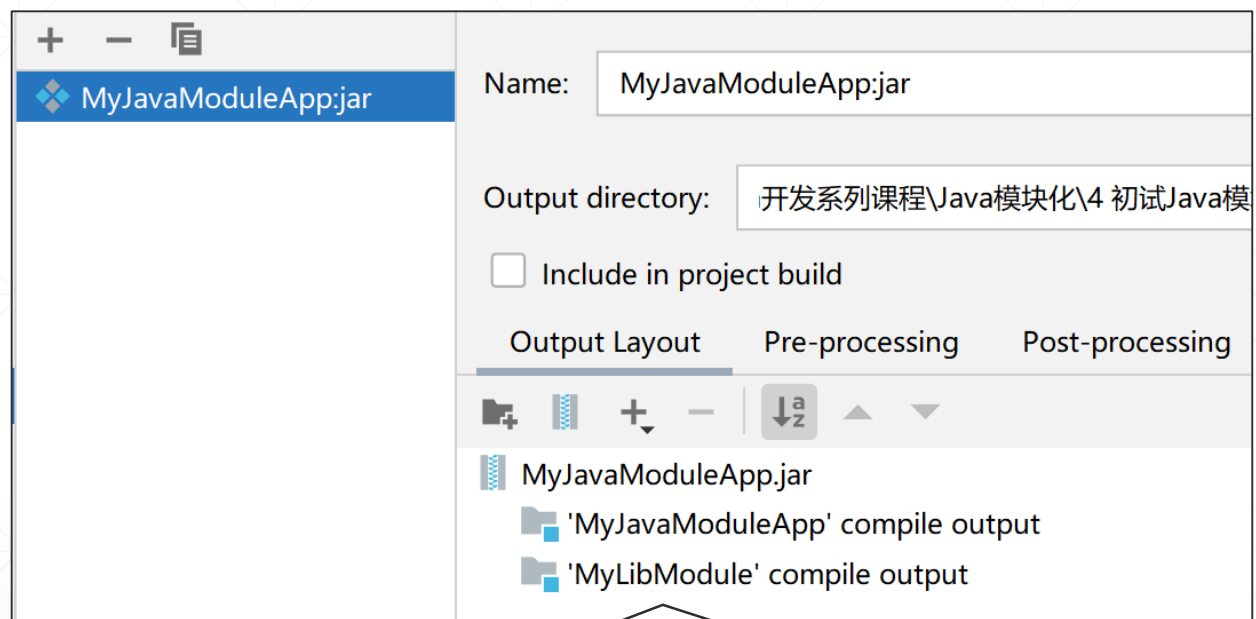


`uses`和`provides`用于定义服务，后面介绍。

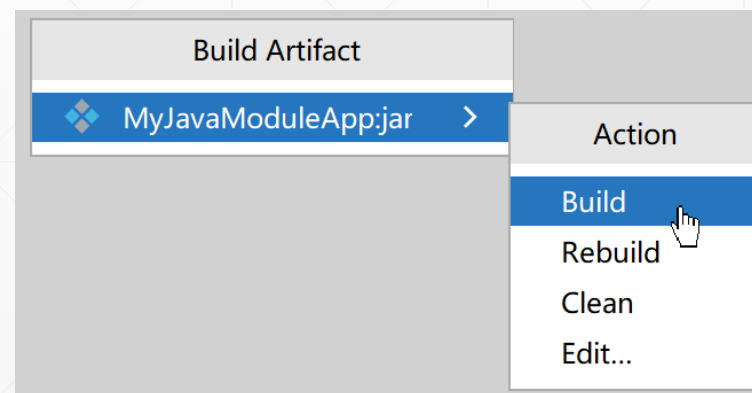
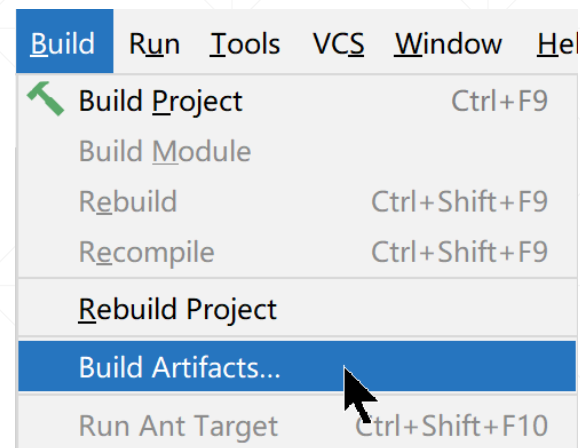
生成模块化Jar包-1



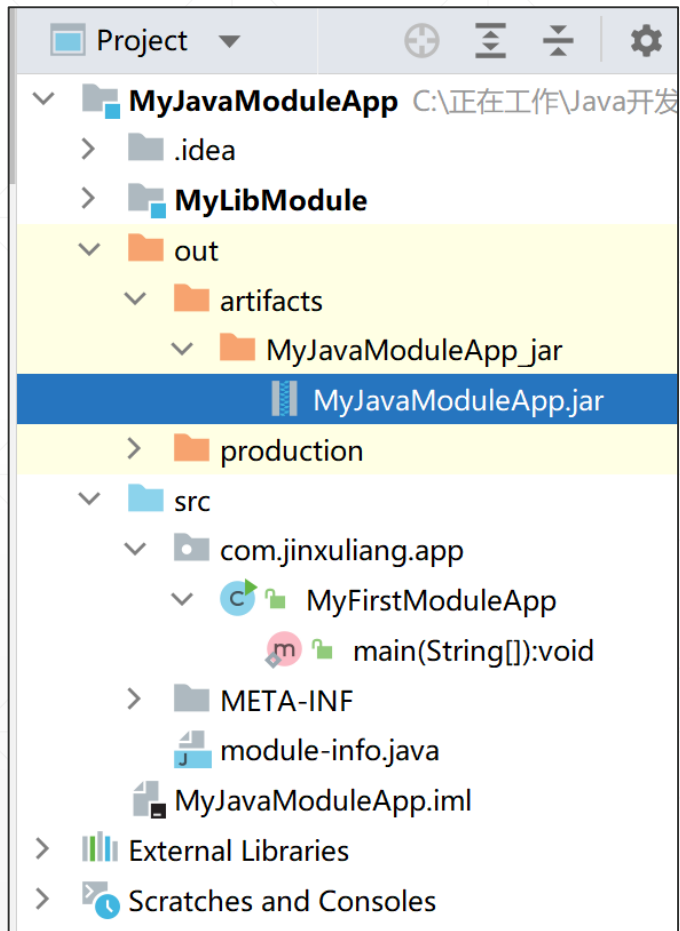
生成模块化Jar包-2



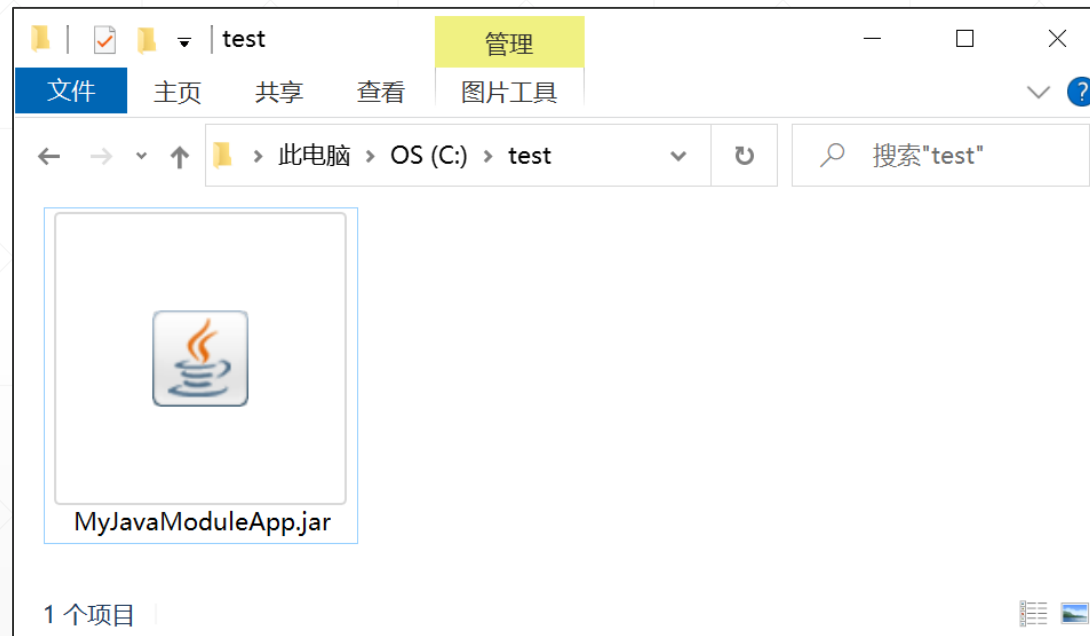
jar包中将包容两个模块的编译
输出结果



Jar包的生成



复制



运行模块化jar包

```
Windows PowerShell
PS C:\test> java -jar .\MyJavaModuleApp.jar
MyLibModule模块中的MyPublicFunc.getInfo()方法被调用
PS C:\test> |
```

仔细查看一下生成的jar包，不难发现，这个jar包中包容了来自于两个模块的.class文件，因此，成为了一个自包容的jar包，可以直接运行。

MyJavaModuleApp.jar - WinRAR

文件(F) 命令(C) 工具(S) 收藏夹(O) 选项(N) 帮助(H)

添加 解压到 测试 查看 删除 查找 向导 信息 扫描病毒 注释 自解压格式

MyJavaModuleApp.jar - ZIP 压缩文件, 解包大小为 1,336 字节

MyJavaModuleApp.jar

- com
 - jinxuliang
 - app
 - META-INF

名称	大小	压缩后大小	类型	修改时间	CRC32
..			文件夹		
META-INF			文件夹	2021/1...	
com			文件夹	2021/1...	
module-info.class	182	132	CLASS 文...	2021/1...	B0EB58...

已经选择 182 字节(1 个文件)

总计 2 文件夹 和 182 字节(1 个文件)

作业



本讲介绍了如何使用IntelliJ IDEA开发模块化Java应用的基本流程，请在PPT的引导下，走完整个示例流程。

找出一个你以前写过的Java程序，或者是其他人写的一个Java程序，尝试着对这个程序进行重构，并将它转换为一个模块化的Java应用。